Module	SE101.3 -OOP with Java – 25.3
Lab sheet Number	07
Areas covered	Polymorphism

Question No 1

Create a class named Calculator that demonstrates method overloading.

Create overloaded methods named add():
- add(int a, int b)
- add(int a, int b, int c)
- add(double a, double b)

Display the result of each method in the main() method.

Question No 02 – Printer Management System

Create a class named Printer that demonstrates method overloading.

Create overloaded methods named print():
- print(String message)
- print(int value)
- print(String message, int copies)

Call all methods from the main() method and display the output.

Question No 03 – Employee Work Management System

Create a superclass named Employee with:
- work()

Create subclasses:
- Developer
- Designer

Requirements:
• Override the work() method in each subclass.
• Display role-specific messages.
• Create objects and demonstrate method overriding.

Question No 04 – Vehicle Engine Control System

Create a superclass named Vehicle with:
• startEngine()

Create subclasses:
• Car
• Motorcycle
• Truck

Requirements:
• Override startEngine() in each subclass.
• Create an array of Vehicle references.
• Demonstrate runtime polymorphism using a loop.

Question No 05 – Cyber Security Authentication System

Create a superclass named AuthenticationMethod with:
• authenticate()

Create subclasses:
• PasswordAuthentication
• FingerprintAuthentication
• FaceRecognitionAuthentication

Requirements:
• Override authenticate() in each subclass.
• Store all objects in an array of AuthenticationMethod references.
• Demonstrate runtime polymorphism.

Question No 06 – Threat Detection System

Create a class named **ThreatDetector** with a method:

```
void detectThreat()
```

that displays a generic threat detection message.

Create the following subclasses:

- MalwareDetector
- PhishingDetector
- IntrusionDetector

Requirements:

- Override the `detectThreat()` method in each subclass.
- Create an array of `ThreatDetector` references.
- Store objects of all subclasses in the array.
- Use a loop to call the `detectThreat()` method for each object.
- Demonstrate runtime polymorphism by processing all threat detectors.

Question No 07 – Security Event Monitoring System

An organization maintains a security monitoring platform.

Create a superclass named **SecurityEvent** with:

- eventID
- eventSource

Requirements:

- Create a constructor to initialize all attributes.
- Create getter methods to display information.
- Create a method named:

```
analyzeEvent()
```

that displays a generic security event analysis message.

Create the following subclasses:

- LoginAttempt
- MalwareAlert

- NetworkAttack

Requirements:

- Override the `analyzeEvent()` method in each subclass.
- Use constructors to initialize all attributes.
- Store all objects in an array of `SecurityEvent` references.
- Use a loop to analyze all events.
- Display event details and analysis results.
- Demonstrate runtime polymorphism by calling the same method through `SecurityEvent` references.